Programming Assignment #2

CS 576 - Computer Networks and Distributed Systems

Instructor: Dr. Wei Wang

Group Members:

Michael Morgan, Maximus Daversa, Paris Cabatit, Justin Cruz

Overview:

Our project implements a simple UDP connectionless communication between a client and server using Python. The client sends a message to the server without establishing a connection, using "Hello" as our default input. The server will listen on a chosen port, receive the message, append a humorous string, and send the modified message back to the client. The client will then display the original and returned message.

System Design:

Server:

The server uses a simple UDP client-server design. The server runs on a localhost port 9999 and waits for incoming UDP messages from the client. Because UDP is connectionless, the server doesn't establish a session before obtaining data. As a message arrives from the client, the server will read the ASCII characters and appends a humorous message at the end, and sends the modified message back to client through their address and port information.

Client:

The client uses UDP to send a single message to the server on localhost port 9999. No connection setup is required for transmission. The client sends an ASCII character message like "Hello," to the server and waits for a reply. Timeout is included so the client isn't waiting forever for a returned message, and if the server isn't running. Once the reply is received, it decodes the ASCII data and prints out the original message and response from the server.

Implementation:

Server:

The server is implemented through Python using the socket library with UDP. This creates a socket and binds it to a localhost (127.0.0.1) on port 9999. The server runs an infinite loop in hopes of retrieving incoming messages through recvfrom(), which provides the client's address. As a message is received, it is decoded from ASCII and then passed to a helper function that appends a humorous message, then sending it back to the client. The server prints both the received message and the reply for logging purposes and continues listening for additional requests.

Client:

The client is implemented through Python as well, using the socket library with UDP. It creates a UDP socket and sets a timeout of 5 seconds in order to avoid waiting indefinitely for a response. The client prepares a message (default message being "Hello"), encodes with ASCII, and sends to the server. Later, it waits for a received message from the server. If a message is sent back from the server, it is decoded and displayed along with the original message. If there is no response back within the timeout period, an error message will display and program exits.

Test and Results:

(Normal Run Screenshot)
(Inputs and outputs)

How To Run?:

1. In Terminal, use the cd command to navigate to the project folder
2. Run server using command 'python udp_server.py'
3. In a separate instance of Terminal, cd to navigate to the same folder
4. In the second, new, terminal run the command 'python udp_client.py [Your Message Here]' The [Your Message Here] is a placeholder, feel free to put anything in as long as it follows ASCII characters

Conclusion:

With this assignment, we were able to utilize a UDP client-server application through the application of python socket programming. We established a server that reads in a message from a client, that's connected to the same port, and returns the message with a humorous message appended to it. The client prepares the message by encoding the message as ASCII prior to sending it to the server, in which the server returns the ASCII message with an appended message. This assignment allowed us to have a working implementation of UDP communication.

Source Code:

```
# Group Programming Assignment #2 - udp_server.py
# Group Members: Michael Morgan, Maximus Daversa, Paris Cabatit,
Justin Cruz
# Instructor: Dr. Wei Wang

# Import statements
import socket

# Constants
SERVER_HOST = "127.0.0.1"
SERVER_PORT = 9999
BUFFER_SIZE = 1024

#Humorous Message to Append
HUMOROUS_JOKES = " What do you call a fish without eyes? A fsh."

# Append humorous suffix to the received message
def get_reply(message: str) -> str:
    return f"{message}{HUMOROUS_JOKES}"

def main():
    #Create a UDP socket and bind it to the set host and port, and
listen for message
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        sock.bind((SERVER_HOST, SERVER_PORT))
        print(f"[SERVER] Listening on {SERVER_HOST}:{SERVER_PORT}
(UDP)  -  Ctrl+C to stop\n")

        # Keep listening for incoming messages and reply by adding
the humorous suffix to
        #The original message, and print the received message and the
```

```
reply being sent back to the client
        while True:
            data, client_addr = sock.recvfrom(BUFFER_SIZE)
            message = data.decode("ascii")
            reply   = get_reply(message)

            print(f"[SERVER] Received from {client_addr}:
'{message}'")
            print(f"[SERVER] Sending reply : '{reply}'\n")

            sock.sendto(reply.encode("ascii"), client_addr)

main()

# Group Programming Assignment #2 - udp_client.py
# Group Members: Michael Morgan, Maximus Daversa, Paris Cabatit,
Justin Cruz
# Instructor: Dr. Wei Wang

# Import statements
import socket
import sys

# Constants
SERVER_HOST = "127.0.0.1"
SERVER_PORT = 9999
BUFFER_SIZE = 1024
TIMEOUT_SEC = 5

def main():
    # Accept an optional command-line message, default to "Hello"
    message = " ".join(sys.argv[1:]) if len(sys.argv) > 1 else
"Hello"

    #Create a UDP socket and set the timeout interval for
communication with the server
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        sock.settimeout(TIMEOUT_SEC)

        # Send the message to the server
        sock.sendto(message.encode("ascii"), (SERVER_HOST,
SERVER_PORT))
        print(f"\n[CLIENT] Sent to server   : \"{message}\"")

        # Receive the reply from the server, with error handling for
timeouts
        try:
```

```
            data, server_addr = sock.recvfrom(BUFFER_SIZE)

        #If a timeout is reached, print an error message and exit
        except socket.timeout:
            print("[CLIENT] ERROR: No response from server (timeout).
Is udp_server.py running?")
            return

        # Decode the reply and print it back to the user
        reply = data.decode("ascii")
        print(f"[CLIENT] Reply received  : \"{reply}\"")
        print(f"\n{'='*60}")
        print(f"You sent    : {message}")
        print(f"Server says : {reply}")
        print(f"{'='*60}\n")

main()
```